INTRODUCTION

This project focuses on analysing current job market trends on LinkedIn by examining which technical and non-technical skills are demanded across different locations in India. Job postings are a rich, real-time source of information about the expectations of employers, but they are unstructured and spread across thousands of web pages. To address this, the project uses Python-based web scraping to automatically collect job postings, extract relevant attributes such as job title, company, location and required skills, and then performs data cleaning and visualisation. The final outcome is a clear picture of the most demanded roles and skills in top locations, helping students and job seekers align their learning with real-world requirements.

ABSTRACT

In this project, job data is scraped from LinkedIn using the public jobs-guest API endpoints. The script first collects multiple pages of job listings, extracts unique job IDs, and then queries the detailed job posting for each ID. For every job, the title, company name, city and full job description are captured. A predefined master list of skills covering software development, data, cloud, security, management, finance, HR and soft skills is used to match skills from the job description using regular expressions. The data is cleaned using pandas, exported to Excel, and then analysed to build a skill–location matrix. A heatmap visualises the frequency of top skills across top locations, while a pie chart displays the top ten most frequently appearing job roles. Together, these outputs provide insight into which skills and job roles are most in demand in the scraped dataset.

TOOLS USED

1. **Python**: The primary programming language used to implement the entire workflow from scraping to analysis and visualisation.
2. **Requests**: Used to send HTTP GET requests to LinkedIn's jobs-guest URLs and retrieve the raw HTML/JSON content for both job lists and individual job postings.
3. **BeautifulSoup (bs4)**: Employed to parse the HTML content, locate specific tags such as job title, company, location, and description, and extract clean text from the markup.

4. **Regular Expressions (re module)**: Used to match skills from the master skill list inside the job description text using word-boundary patterns, ensuring accurate detection of complete skills.

5. **Pandas**: Utilised for building tabular datasets, cleaning and transforming data, handling missing values, removing duplicates, creating pivot tables, and exporting results to Excel.

6. **Seaborn**: Used to create a heatmap that visualises how frequently each top skill appears in each top location, providing an easy-to-read summary of the skill distribution.

7. **Matplotlib**: Provides the underlying plotting framework for Seaborn and is also used directly to generate the pie chart of the top 10 job roles.

8. **Microsoft Excel (output file: linkedin_jobss.xlsx)**: Acts as the storage and reporting format for the cleaned dataset so that the results can be inspected, shared, or further analysed outside Python.

STEPS INVOLVED IN BUILDING THE PROJECT

1. **Defining the Master Skill List**: A comprehensive master_skills list was manually prepared, containing technical, managerial, financial, HR, marketing and soft skills. All skills are in lowercase and sorted in descending order of length. Sorting by length ensures that longer phrases like "machine learning" are matched before shorter overlapping words, reducing partial or incorrect matches during pattern search.

2. **Scraping Job IDs from LinkedIn**: A base LinkedIn jobs-guest URL was used with pagination controlled by the start parameter. For each page, the script sent a request and parsed the response with BeautifulSoup. Every <li> element representing a job card was processed to extract the data-entity-urn attribute from the div.base-card. The job ID was derived from this URN and appended to a list. After pagination, duplicate job IDs were removed to ensure that each job was processed only once.

3. **Fetching Detailed Job Information**: For each unique job ID, a second request was made to a job-posting specific URL (/jobs/api/jobPosting/{job_id}). The detailed page was parsed to extract four key elements: job title (h2.top-card-layout__title), company

name (span.topcard__flavor), location (span.topcard__flavor--bullet) and job description (div.description__text). Only records where all these fields were successfully found were retained to maintain data quality.

4. **Extracting and Matching Skills from Job Descriptions**: The job description text was converted to lowercase and cleaned. For each skill in master_skills, a regular expression with word boundaries (\bskill\b) was used to search within the description. If a match was found, that skill was added to a temporary list. After scanning all skills, duplicates were removed, the list was sorted, and the selected skills were joined into a comma-separated string. This string represents the set of skills demanded by that particular job posting.

5. **Building the Initial Dataset**: The extracted job ID, title, company, city (derived from the first part of the location string) and matched skills string were appended to their respective lists. These lists were then combined into a pandas DataFrame named job_dataset with columns: “Job ID”, “Job Title”, “Company”, “Location” and “Skills”. A preview and shape of the dataset were printed to ensure that scraping worked as expected.

6. **Data Cleaning and Pre-processing**: Before analysis, several cleaning steps were applied. Extra spaces were stripped from the string columns, and empty strings were converted to NaN. Duplicate rows based on the combination of job ID, title, company and location were dropped to avoid double counting. Rows lacking any critical fields were removed. For the “Skills” column, missing values were filled with the placeholder “Not specified” to indicate that no skills were detected or listed. After cleaning, the final shape and null counts were printed to validate the pre-processing.

7. **Exporting the Cleaned Dataset to Excel**: The cleaned DataFrame was saved as linkedin_jobss.xlsx. This step makes the dataset reusable for further analysis, reporting and sharing. It also acts as a backup so that the scraping process does not have to be repeated for every analysis.

8. **Transforming Skills for Location–Skill Analysis**: To analyse skill demand by location, the skills string for each job was split into a list. For each job, a small record was created for every combination of its location and each of its skills. These records were combined into a new DataFrame (skill_loc_df) with two columns: “Location” and “Skill”.

9. **Creating a Skill–Location Pivot Table**: A pivot table was constructed from skill_loc_df where each row represents a location, each column represents a skill, and the cell values represent the count of that skill in that location. From this matrix, the top 10 locations (by total skill counts) and top 10 skills (by total occurrences) were selected. A sub-matrix (heatmap_df) was created containing only these top locations and skills.
10. **Visualising the Results**: Two main visualisations were generated. First, a heatmap using Seaborn displayed the frequency of each of the top skills in each of the top locations, highlighting where particular skills are highly demanded. Second, the distribution of job roles was examined by counting job titles and plotting a pie chart of the top 10 most frequently appearing job titles. These charts offer an intuitive understanding of which job roles dominate the dataset and which skills are essential in the most active locations.

CONCLUSION

This project successfully demonstrates a complete pipeline for extracting, cleaning and analysing job market data from LinkedIn using Python. By combining web scraping, a curated master skill list and regular expression matching, it becomes possible to automatically derive the skill requirements of a large number of job postings. The cleaned dataset and visualisations reveal which roles are most advertised and which technical and soft skills are in highest demand across different locations. This information can support students, job seekers and training institutes in aligning their learning paths with real-time industry expectations. In future work, the project can be extended by adding time-based analysis, role-specific dashboards, more sophisticated natural language processing techniques and larger, regularly updated datasets, while also carefully respecting platform policies and usage limits.